White Paper

YOLO Models Architecture

Advanced Object Detection Systems

A Comprehensive Technical Analysis of Real-Time Computer Vision Evolution

Prepared by:

Anmol Maheshwari
Neil Mittal

Indian Institute of Technology, Kharagpur

Contents

PART I: YOLOv1 **3**

1 Introduction **3**
1.1 Conceptual Foundation . . . 3

2 Unified Detection Framework **3**
2.1 Algorithmic Overview . . . 3
2.1.1 Residual Blocks and Grid Division . . . 3
2.1.2 Bounding Box Regression . . . 4
2.1.3 Intersection Over Union (IoU) . . . 4
2.1.4 Non-Maximum Suppression (NMS) . . . 4
2.2 Input Preprocessing and Grid Encoding . . . 4

3 Model Architecture and Implementation **6**
3.1 Model Output and Dimensional Analysis (YOLO v1) . . . 6
3.2 Network Architecture Specification . . . 6
3.3 Training Configuration and Hyperparameters . . . 7
3.4 Activation Function Selection . . . 7

4 Loss Function Formulation **9**
4.1 Multi-Component Loss Architecture . . . 9
4.2 Loss Function Rationale and Design Considerations . . . 9

5 Output Parsing and Postprocessing Pipeline **11**
5.1 Confidence Computation and Thresholding . . . 11
5.2 Non-Maximum Suppression Implementation . . . 11

PART II: YOLOv2 **11**

6 Introduction **11**

7 Architectural Enhancements **11**
7.1 Batch Normalization Implementation . . . 11
7.1.1 Benefits of Batch Normalization in YOLOv2 . . . 12
7.2 High-Resolution Classifier . . . 12
7.3 Convolutional Architecture with Anchor Boxes . . . 13
7.4 Dimension Clusters via K-Means . . . 13
7.5 Direct Location Prediction . . . 14
7.6 Fine-Grained Feature Integration . . . 14
7.7 Multi-Scale Training . . . 14

8 Training Methodology **14**
8.1 Classification Network Training . . . 14
8.2 Detection Network Adaptation . . . 15

9 Hierarchical Classification Mechanism **15**
9.1 Joint Training Strategy . . . 15
9.2 WordNet-Based Hierarchy . . . 15

10 Performance Characteristics **15**
10.1 Detection Capabilities . . . 15
10.2 Computational Efficiency . . . 16

PART III: YOLOv3 **16**

11 Introduction **16**

12 Bounding Box Prediction Mechanism **17**
12.1 Logistic Regression for Objectness Scoring . . . 17
12.2 Responsibility Assignment Mechanism . . . 17
12.3 Coordinate Prediction through Offset Parameterization . . . 18

13 Class Prediction System **18**
13.1 Multi-Label Classification Framework . . . 18
13.2 Independent Logistic Classifiers . . . 18
13.3 Binary Cross-Entropy Loss Formulation . . . 19

14 Multi-Scale Detection Architecture **19**
14.1 Feature Pyramid Network Integration . . . 19
14.2 Upsampling and Feature Concatenation . . . 19
14.3 Recursive Scale Prediction . . . 20

15 Darknet-53 Feature Extractor **20**
15.1 Architectural Hybridization . . . 20
15.2 Layer Composition and Connectivity . . . 20
15.3 Computational Efficiency Metrics . . . 21

16 Performance Characteristics and Benchmark Results **21**
16.1 Inference Speed and Accuracy Trade-off . . . 21

You Only Look Once (YOLO): A Unified Framework for Real-Time Object Detection

Abstract

This white paper provides a comprehensive examination of the You Only Look Once (YOLO) object detection framework, which represents a shift in computer vision by reframing object detection as a single-stage regression problem. Unlike traditional approaches that separate region proposal and classification into distinct stages, YOLO employs a single convolutional neural network that directly predicts bounding boxes and class probabilities in a single forward pass. This unified architecture enables real-time processing while maintaining competitive accuracy. The paper details the algorithmic foundations, architectural implementations across YOLO versions, training methodologies, and mathematical formulations that underpin this influential detection framework.

`https://github.com/Anmol-Maheshwari30/yolov1`

YOLOv1

1 Introduction

1.1 Conceptual Foundation

You Only Look Once (YOLO) represents a reimagining of object detection to a regression-based approach. Traditional object detection systems employ a two-stage process: first generating region proposals that may contain objects, then classifying these regions and refining their boundaries. YOLO performs object detection as a single-stage regression problem, where a single convolutional neural network processes the entire image and directly predicts bounding box coordinates and class probabilities in one forward pass. This architectural simplification yields significant advantages in computational efficiency while maintaining detection performance, making it suitable for real-time applications.

The YOLO algorithm processes input images through a grid-based detection system. The fundamental principle involves dividing the input image into an $S \times S$ grid of equally sized cells. Each grid cell assumes responsibility for detecting objects whose geometric centers fall within its spatial boundaries. For each cell, the network predicts a fixed number B of bounding boxes alongside C conditional class probabilities. This unified framework enables simultaneous prediction of object location and category, allowing the entire image to be processed in a single computational pass. The integration of localization and classification tasks within a single network architecture represents a significant advancement in detection efficiency and implementation simplicity.

2 Unified Detection Framework

2.1 Algorithmic Overview

The YOLO detection algorithm operates through four interconnected methodological components that collectively enable efficient object localization and classification:

2.1.1 Residual Blocks and Grid Division

The input images are divided into N grid cells of uniform dimensions. Each grid cell is responsible for localizing and predicting the class of objects whose centroids fall within its spatial boundaries and the probability or confidence value

is estimated. The grid-based approach provides a structured framework for distributing detection tasks across spatial regions, enabling parallel processing of potential object locations.

2.1.2 Bounding Box Regression

YOLO employs bounding box regression to determine the rectangular regions that highlight all detectable objects within an image. A single image may contain n bounding boxes corresponding to various object instances. YOLO determines the geometric attributes of these bounding boxes through a unified regression module, where each bounding box is represented by a vector Y encoding its spatial and probabilistic characteristics.

2.1.3 Intersection Over Union (IoU)

During object detection a single object can generate multiple bounding boxes. To solve this, YOLO employs Intersection Over Union (IoU) as a quantitative measure of bounding box relevance. IoU, expressed as a value between 0 and 1 and quantifies the spatial overlap between predicted and ground truth bounding boxes. The algorithm computes IoU as the area of intersection divided by the area of union between bounding box pairs. The IOU value can be specified as a threshold parameter, predictions from grid cells exhibiting values below this threshold are discarded.

2.1.4 Non-Maximum Suppression (NMS)

While IoU thresholding eliminates many irrelevant detections, scenarios persist where multiple high-overlap bounding boxes exceed the IoU threshold while representing the same object instance. To address this redundancy, YOLO implements Non-Maximum Suppression (NMS), a post-processing algorithm that retains only the bounding box with the highest probability detection score among overlapping candidates. This suppression mechanism ensures that each object instance receives a single, high-confidence detection, reducing false positives and computational redundancy in the final output.

2.2 Input Preprocessing and Grid Encoding

In YOLO version 1, input images undergo resizing to a fixed 448×448 pixel resolution. This normalized input is then partitioned into a grid of $S \times S$ cells, with $S = 7$ in the initial implementation. Each grid cell predicts B bounding boxes, with each bounding box represented by five numerical parameters:

$$(x_m, y_m, w_m, h_m, p_{\text{obj}}) \tag{1}$$

Where x and y denote the normalized center coordinates of the bounding box relative to the containing grid cell, with values between 0 and 1. Parameters w and h represent the width and height of the bounding box, expressed relative to the entire image dimensions and similarly normalized. The objectness score p_{obj} quantifies detection confidence, defined mathematically as the product of the probability that an object exists within the bounding box and the predicted Intersection over Union (IoU) between the predicted box and its corresponding ground truth.

Along with the bounding box parameters, each grid cell predicts C conditional class probabilities $P(\text{class}_i \mid \text{object})$. During training, these probabilities are encoded using representation within the target vectors. This process transforms image-level ground-truth annotations into grid-specific targets: the grid cell containing an object's geometric center shows the complete object representation including bounding box parameters and class label, while cells lacking object centers show an objectness value of zero.

Listing 1: YOLOv1 Model Definition using Keras and ResNet50 Backbone

```
import os
import cv2
import numpy as np
import xml.etree.ElementTree as ET

img_size = 224
#reduced from 448 to reduce memory
s = 7
```

```
b = 2

folder_path = '/path/to/your/VOC2007/directory'

classes= ["aeroplane","bicycle","bird","boat","bottle","bus","car","cat",
    "chair","cow","diningtable","dog","horse","motorbike","person",
    "pottedplant","sheep","sofa","train","tvmonitor"]

def load_image_ids (voc_root):
    path = os.path.join(voc_root , "ImageSets" , "Main", "trainval.txt")
    with open(path) as f:
        return[line.strip() for line in f.readlines()]

def load_image(voc_root , image_id):
    path = os.path.join(voc_root ,"JPEGImages" , image_id + ".jpg")
    img = cv2.imread(path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (img_size, img_size))
    return preprocess_input(img.astype(np.float32))

def parse_annotation(voc_root, image_id):
    path = os.path.join(voc_root, "Annotations" , image_id + ".xml")
    tree = ET.parse(path)
    root = tree.getroot()

    width = int(root.find("size/width").text)
    height = int(root.find("size/height").text)

    boxes, labels = [] , []

    for obj in root.findall("object"):
        label = obj.find("name").text
        if label not in classes:
          continue
        box = obj.find("bndbox")
        xmin = (float(box.find("xmin").text)/width)*img_size
        ymin = (float(box.find("ymin").text)/height)*img_size
        xmax = (float(box.find("xmax").text)/width)*img_size
        ymax = (float(box.find("xmax").text)/height)*img_size
        boxes.append([xmin, ymin, xmax, ymax])
        labels.append(label)

    return boxes, labels

def encode_yolo_target(boxes, labels, img_size=224, S=7, B=2, C=20):
    target = np.zeros((S, S, B*5 + C), dtype=np.float32)

    cell_size = img_size / S

    for box, label in zip(boxes, labels):
        xmin, ymin, xmax, ymax = box
        x_center = (xmin + xmax) / 2.0
        y_center = (ymin + ymax) / 2.0
        w = xmax - xmin
        h = ymax - ymin

        i = min(int(y_center / cell_size), s-1)
        j = min(int(x_center / cell_size),s-1)

        x_cell = (x_center - j * cell_size) / cell_size
        y_cell = (y_center - i * cell_size) / cell_size

        w_norm = w / img_size
        h_norm = h / img_size
```

```
        class_id = classes.index(label)

        target[i, j, 0:5] = [x_cell, y_cell, w_norm, h_norm, 1.0]
        target[i, j, B*5 + class_id] = 1.0

    return target
```

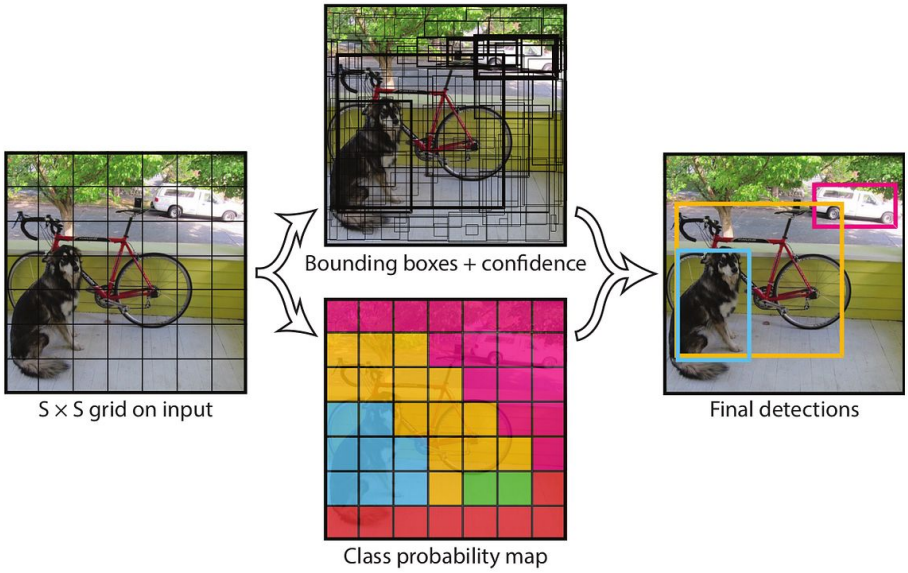


Figure 1: The Model: Detection works as a regression problem

3 Model Architecture and Implementation

3.1 Model Output and Dimensional Analysis (YOLO v1)

For YOLO version 1 with parameter values $S = 7$, $B = 2$, and $C = 20$ (corresponding to the Pascal VOC dataset), the dimensional characteristics of the output tensor can be precisely calculated. Each grid cell predicts $B \times 5 + C = 2 \times 5 + 20 = 30$ distinct numerical values. Consequently, the complete output tensor assumes the dimensionality $S \times S \times (B \cdot 5 + C) = 7 \times 7 \times 30$.

During inference operations, each of the two bounding boxes per cell generates an independent objectness score. The framework computes class-specific confidence scores through element-wise multiplication of conditional class probabilities with corresponding bounding box objectness scores:

$$\text{class confidence}_{i,\text{box}} = P(\text{class}_i \mid \text{object}) \times p_{\text{obj,box}} \tag{2}$$

Following confidence computation, the algorithm applies Non-Maximum Suppression (NMS) to eliminate redundant bounding boxes exhibiting substantial spatial overlap, thereby refining the final detection set.

3.2 Network Architecture Specification

YOLO version 1 employs a convolutional neural network architecture conceptually inspired by GoogleNet's inception modules, supplemented by fully connected layers responsible for bounding box and class probability regression. The architectural composition includes multiple convolutional and max-pooling layers (specifically 24 convolutional layers in version 1) that progressively transform input images into high-level feature representations, culminating in a final feature map of dimensionality $7 \times 7 \times 1024$.

The network incorporates specialized convolutional branches modeled after inception modules:

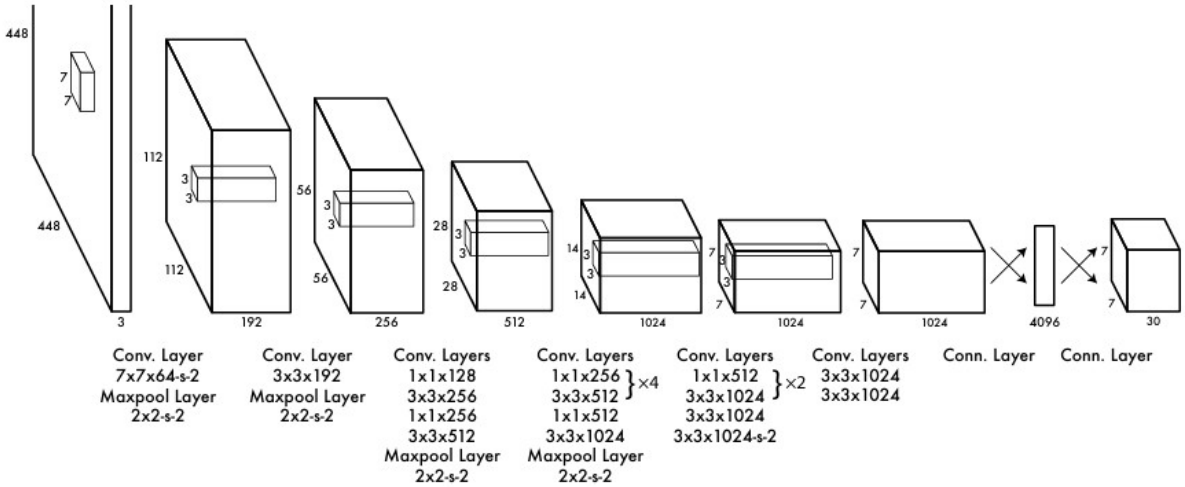


Figure 2: Network Architecture

- 1×1 convolutions capture localized features while simultaneously reducing channel depth for computational efficiency.
- 3×3 convolutions extract medium-scale spatial features with balanced receptive fields.
- 5×5 convolutions capture larger, more global features encompassing broader contextual information.
- 3×3 maximum pooling operations introduce spatial invariance and emphasize dominant feature activations.

The detection pipeline comprises three functional components: a backbone network serving as a feature extractor that transforms pixel data into semantically meaningful representations; intermediate layers that reshape spatial features into vectorized form; and a final regression head that generates bounding box coordinates and class probabilities.

3.3 Training Configuration and Hyperparameters

YOLO version 1 undergoes training for 135 complete epochs utilizing combined datasets from PASCAL VOC 2007 and 2012. The optimization process employs a batch size of 64 samples, momentum parameter of 0.9, and weight decay regularization of 0.0005.

A progressive learning rate schedule stabilizes training and facilitates convergence:

- Initial epochs gradually increase learning rate from 10^{-3} to 10^{-2} to prevent gradient instability that may cause model divergence.
- Primary training phase maintains 10^{-2} learning rate for 75 epochs.
- Intermediate refinement employs 10^{-3} learning rate for 30 epochs.
- Final fine-tuning utilizes 10^{-4} learning rate for 30 epochs.

To mitigate overfitting and enhance generalization capability, the training regimen incorporates data augmentation techniques and includes a dropout layer with retention probability of 0.5 within the fully connected network segments.

3.4 Activation Function Selection

The YOLO architecture employs distinct activation functions across network layers. All intermediate layers utilize the leaky rectified linear unit (Leaky ReLU) activation, while the final output layer employs linear activation. The Leaky ReLU function is mathematically defined as:

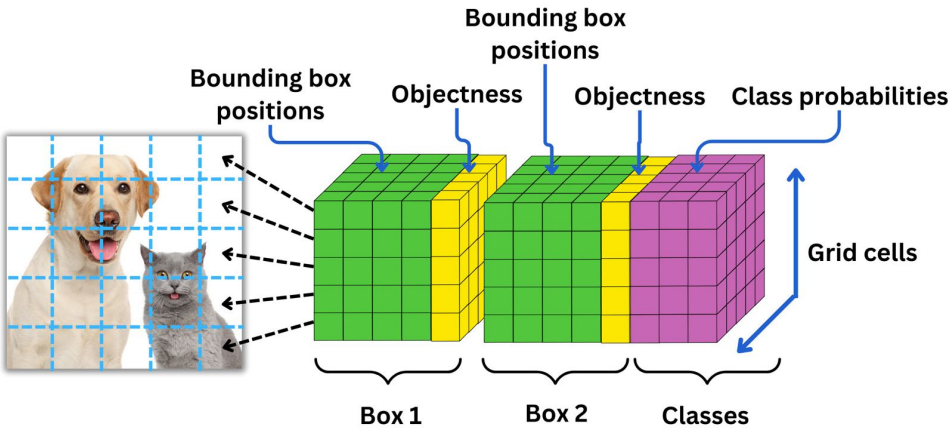


Figure 3: Description of the image.

$$\phi(x) = \begin{cases} x, & x > 0 \\ 0.1x, & \text{otherwise} \end{cases} \tag{3}$$

This piecewise-linear activation mitigates the vanishing gradient problem associated with standard ReLU while maintaining computational efficiency during forward and backward propagation.

Listing 2: YOLOv1 Model Definition using Keras and ResNet50 Backbone

```
import tensorflow as tf
import keras
from keras import layers, models
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.applications.resnet50 import preprocess_input

def conv(x , f, k , s= 1):
    x = layers.Conv2D(f, k , s, padding = "same" , use_bias = False)(x)
    x = layers.BatchNormalization()(x)
    return layers.LeakyReLU(0.1)(x)

def yolov1():
  base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
  base_model.trainable = False

  x = base_model.output

  x = layers.Conv2D(1024,3, padding = "same" , use_bias = False)(x)
  x = layers.BatchNormalization()(x)
  x = layers.LeakyReLU(0.1)(x)

  x = layers.Conv2D(1024,3, padding = "same" , use_bias = False)(x)
  x = layers.BatchNormalization()(x)
  x = layers.LeakyReLU(0.1)(x)


  x = layers.Flatten()(x)
  x = layers.Dense(4096)(x)
  x = layers.LeakyReLU(0.1)(x)
  x = layers.Dropout(0.5)(x)

  x = layers.Dense(7*7*30, activation = "sigmoid")(x)
  out = layers.Reshape((7,7,30))(x)

  return models.Model(base_model.input, outputs= out)

if __name__ == "__main__":
```

```
    model = yolov1()
    model.summary()
```

4 Loss Function Formulation

4.1 Multi-Component Loss Architecture

YOLO implements a composite loss function comprising weighted sum-of-squared-error terms that balance contributions from localization accuracy, object confidence estimation, and classification correctness. Two hyperparameters modulate the relative influence of different loss components:

- λ_{coord}: Weighting factor for bounding box coordinate loss, typically set to 5.
- λ_{noobj}: Weighting factor for confidence loss in grid cells lacking objects, typically set to 0.5.

Let S denote grid size (7 in v1), B represent bounding boxes per cell (2 in v1), C indicate class count, $\mathbb{1}_i^{\text{obj}}$ serve as an indicator function returning 1 when an object appears in cell i, and $\mathbb{1}_{ij}^{\text{obj}}$ indicate when the j-th bounding box predictor in cell i demonstrates responsibility for an object (exhibiting highest IoU with ground truth).

The complete loss function L:

$$
\begin{aligned}
L = \lambda_{\text{coord}} & \sum_{i=1}^{S^2} \sum_{j=1}^{B} \mathbf{1}_{ij}^{\text{obj}} \left[(x_{ij} - \hat{x}_{ij})^2 + (y_{ij} - \hat{y}_{ij})^2 \right] \\
& + \lambda_{\text{coord}} \sum_{i=1}^{S^2} \sum_{j=1}^{B} \mathbf{1}_{ij}^{\text{obj}} \left[(\sqrt{w_{ij}} - \sqrt{\hat{w}_{ij}})^2 + (\sqrt{h_{ij}} - \sqrt{\hat{h}_{ij}})^2 \right] \\
& + \sum_{i=1}^{S^2} \sum_{j=1}^{B} \mathbf{1}_{ij}^{\text{obj}} \left(p_{\text{obj},ij} - \hat{p}_{\text{obj},ij} \right)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=1}^{S^2} \sum_{j=1}^{B} \mathbf{1}_{ij}^{\text{noobj}} \left(p_{\text{obj},ij} - \hat{p}_{\text{obj},ij} \right)^2 \\
& + \sum_{i=1}^{S^2} \mathbf{1}_{i}^{\text{obj}} \sum_{c=1}^{C} \left(P(c \mid \text{object})_i - P(\widehat{c \mid \text{object}})_i \right)^2 .
\end{aligned}
\tag{4}
$$

4.2 Loss Function Rationale and Design Considerations

The loss function incorporates several deliberate design decisions that optimize detection performance:

- **Square Root Transformation**: This reduces the influence of large bounding boxes on the loss, as width and height values show variation across object scales. This ensures balanced gradient contributions from both small and large bounding boxes.
- **Responsibility Assignment**: The indicator function $\mathbf{1}_{ij}^{\text{obj}}$ ensures that only the bounding box predictor demonstrating highest IoU with a ground-truth object contributes to localization and objectness losses for that specific object.
- **Negative Example Downweighting**: The λ_{noobj} parameter reduces loss contributions from bounding boxes that correctly predict object absence, preventing negative examples from affecting the loss function.
- **Selective Classification Penalization**: Classification error penalties apply exclusively to grid cells containing object centers, focusing learning capacity on relevant detection contexts.

Listing 3: YOLOv1 Model Definition using Keras and ResNet50 Backbone

```
import tensorflow as tf

def bbox_iou(boxes1, boxes2):
    boxes1_xy = boxes1[..., 0:2]
```

```
    boxes1_wh = boxes1[..., 2:4]
    boxes1_mins = boxes1_xy - boxes1_wh / 2.
    boxes1_maxes = boxes1_xy + boxes1_wh / 2.

    boxes2_xy = boxes2[..., 0:2]
    boxes2_wh = boxes2[..., 2:4]
    boxes2_mins = boxes2_xy - boxes2_wh / 2.
    boxes2_maxes = boxes2_xy + boxes2_wh / 2.

    intersect_mins = tf.maximum(boxes1_mins, boxes2_mins)
    intersect_maxes = tf.minimum(boxes1_maxes, boxes2_maxes)

    intersect_wh = tf.maximum(intersect_maxes - intersect_mins, 0.)
    intersect_area = intersect_wh[..., 0] * intersect_wh[..., 1]

    boxes1_area = tf.maximum(boxes1_wh[..., 0] * boxes1_wh[..., 1], 1e-10)
    boxes2_area = tf.maximum(boxes2_wh[..., 0] * boxes2_wh[..., 1], 1e-10)

    union_area = boxes1_area + boxes2_area - intersect_area
    iou = intersect_area / tf.maximum(union_area, 1e-10)

    return iou

def yolo_loss(y_true, y_pred):

    obj_mask = y_true[..., 4:5]
    no_obj_mask = 1.0 - obj_mask

    true_xy = y_true[..., 0:2]
    pred_xy = y_pred[..., 0:2]
    true_wh = y_true[..., 2:4]
    pred_wh = y_pred[..., 2:4]

    pred_wh = tf.maximum(pred_wh, 1e-6)
    xy_loss = tf.reduce_sum(tf.square(true_xy - pred_xy), axis=-1, keepdims=True)

    wh_loss = tf.reduce_sum(tf.square(tf.sqrt(true_wh + 1e-6) - tf.sqrt(tf.abs(pred_wh) + 1e-6)),
        axis=-1, keepdims=True)
    coord_loss = 5.0 * tf.reduce_sum(obj_mask * (xy_loss + wh_loss))

    iou_scores = bbox_iou(y_true[..., 0:4], y_pred[..., 0:4])
    true_conf_target = iou_scores[..., tf.newaxis]

    pred_conf = y_pred[..., 4:5]

    conf_loss_obj = tf.reduce_sum(obj_mask * tf.square(true_conf_target - pred_conf))

    conf_loss_noobj = 0.5 * tf.reduce_sum(no_obj_mask * tf.square(tf.zeros_like(true_conf_target) -
        pred_conf))

    true_class = y_true[..., 10:]
    pred_class = y_pred[..., 10:]
    class_loss = tf.reduce_sum(obj_mask * tf.square(true_class - pred_class))

    total_loss = (coord_loss + conf_loss_obj + conf_loss_noobj + class_loss)

    batch_size = tf.cast(tf.shape(y_true)[0], tf.float32)
    total_loss = tf.where(tf.math.is_finite(total_loss), total_loss, tf.zeros_like(total_loss))
    return total_loss / tf.cast(tf.shape(y_true)[0], tf.float32)
```

5 Output Parsing and Postprocessing Pipeline

5.1 Confidence Computation and Thresholding

Following network inference, the framework computes class-specific confidence scores through element-wise multiplication of objectness probabilities p_{obj} with conditional class probabilities $P(\text{class}_i \mid \text{object})$. Bounding boxes generating confidence scores below a predefined threshold undergo immediate elimination, reducing computational burden in subsequent processing stages.

5.2 Non-Maximum Suppression Implementation

The algorithm applies class-specific Non-Maximum Suppression (NMS) to remaining bounding boxes, systematically eliminating detections exhibiting high spatial overlap (quantified by IoU) while preserving the highest-scoring candidate for each object instance. This suppression mechanism ensures that each detectable object receives a single, high-confidence bounding box prediction in the final output.

YOLOv2 (YOLO 9000)

6 Introduction

YOLOv2, also designated as YOLO 9000, represents a substantial advancement in the YOLO object detection paradigm, addressing limitations of the original architecture while introducing novel mechanisms for improved performance. The system achieves the remarkable capability of detecting over 9,000 distinct object categories while simultaneously maintaining high localization accuracy and classification precision. This expansion in detection capability is accomplished through architectural refinements and innovative training strategies that enhance both detection speed and accuracy. The framework builds upon the foundational YOLO principles while introducing batch normalization, anchor box mechanisms, dimension clustering, and multi-scale training to create a more robust and versatile detection system.

7 Architectural Enhancements

7.1 Batch Normalization Implementation

YOLOv2 extensively incorporates batch normalization throughout its convolutional layers, resulting in significant improvements in training convergence and model stability. The application of batch normalization across all convolutional layers yields an approximate 2% improvement in mean Average Precision (mAP). This regularization effect proves sufficiently robust to permit removal of dropout layers without inducing overfitting, simplifying the network architecture while enhancing performance.

Batch normalization operates by normalizing layer inputs across each mini-batch during training. For a mini-batch $\mathcal{B}$ of size m_B, the algorithm computes mean and variance statistics for each feature channel:

$$\mu_{\mathcal{B}} = \frac{1}{m_B} \sum_{i=1}^{m_B} x^{(i)} \tag{5}$$

$$\sigma_{\mathcal{B}}^2 = \frac{1}{m_B} \sum_{i=1}^{m_B} (x^{(i)} - \mu_{\mathcal{B}})^2 \tag{6}$$

Each activation is subsequently normalized according to:

$$\hat{x}^{(i)} = \frac{x^{(i)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \tag{7}$$

This normalized output undergoes a learned affine transformation:

$$z^{(i)} = \gamma \otimes \hat{x}^{(i)} + \beta \tag{8}$$

where γ and β represent trainable per-channel parameters and $\otimes$ denotes element-wise scaling operations.

During training phases, batch normalization utilizes current mini-batch statistics $(\mu_{\mathcal{B}}, \sigma^2_{\mathcal{B}})$, while during inference operations these are replaced with running population estimates accumulated throughout training. This transition ensures stable, deterministic predictions during deployment.

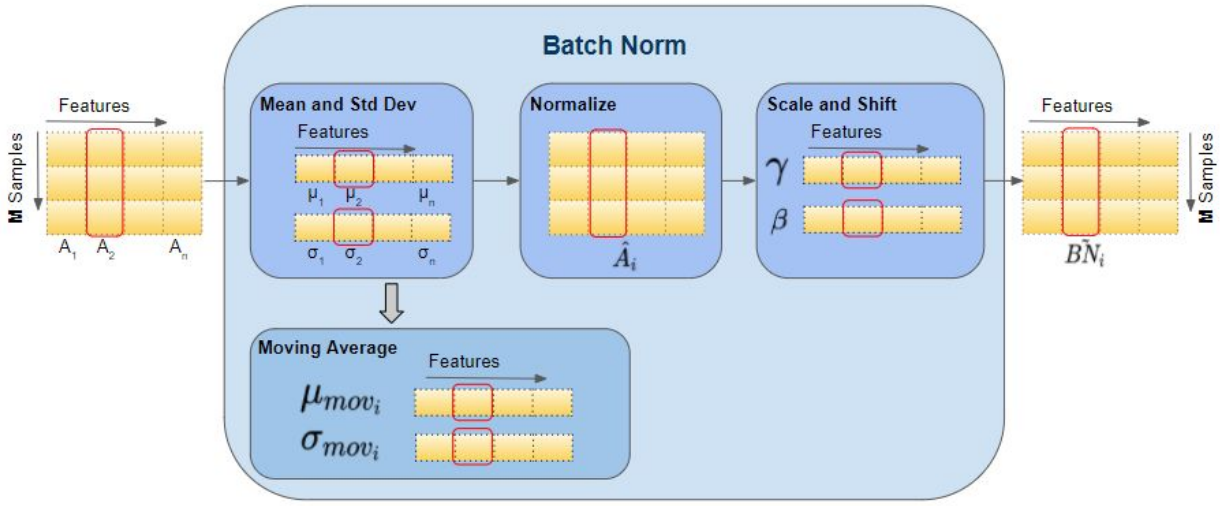


Figure 4: Batch Normalization

7.1.1 Benefits of Batch Normalization in YOLOv2

The integration of batch normalization provides several distinct advantages within the YOLOv2 architecture:

- **Stabilized Feature Distributions**: By normalizing channel activations, batch normalization reduces internal covariate shift, enabling the convolutional backbone to learn more reliable spatial features for objectness estimation, bounding-box regression, and classification tasks.
- **Accelerated and Stable Optimization**: Normalized activations permit higher learning rates while reducing gradient oscillations during training of localization and objectness components, thereby improving convergence characteristics of the multi-part YOLO loss function.
- **Preserved Representational Capacity**: The scale and shift parameters (γ, β) allow the network to recover optimal activation distributions post-normalization, ensuring no degradation in expressive power or representational capability.
- **Enhanced Generalization**: Batch normalization introduces stochasticity through mini-batch-dependent statistics, functioning as an implicit regularizer that proves particularly beneficial for large-scale detection models trained on diverse image distributions.

7.2 High-Resolution Classifier

YOLOv2 employs a high-resolution classifier strategy to enhance detection precision. The system utilizes a classifier pre-trained on the ImageNet dataset, which undergoes fine-tuning at full 448×448 resolution for 10 epochs. This extended fine-tuning period allows convolutional filters to adjust their representations for higher-resolution inputs, resulting in an approximate 4% improvement in mean Average Precision (mAP).

This resolution adaptation addresses limitations of earlier approaches that transitioned directly from lower-resolution classification networks to detection tasks. By providing the network with adequate time to adjust filter weights to the increased spatial dimensions, YOLOv2 achieves superior feature extraction capabilities that translate directly into improved detection performance.

7.3 Convolutional Architecture with Anchor Boxes

YOLOv2 fundamentally restructures its detection mechanism through the introduction of anchor boxes and convolutional adaptations. The architectural modifications include:

- **Fully Connected Layer Removal**: YOLOv2 eliminates fully connected layers present in the original YOLO architecture, replacing them with convolutional operations that maintain spatial coherence throughout the network.
- **Pooling Layer Reduction**: The system removes one pooling layer to increase the resolution of convolutional outputs, enhancing the network's capacity to detect smaller objects.
- **Input Resolution Adjustment**: YOLOv2 processes 416×416 input images rather than the 448×448 resolution used in YOLOv1. This adjustment produces feature maps with odd spatial dimensions (13×13), ensuring a single center cell that facilitates object localization.
- **Downsampling Factor**: The network downsamples input images by a factor of 32, transforming 416×416 inputs into 13×13 feature maps while preserving spatial relationships.
- **Decoupled Prediction Mechanism**: YOLOv2 separates class prediction from spatial location estimation, predicting class probabilities and objectness scores independently for each anchor box.

These architectural changes yield measurable performance improvements: while the original YOLO achieved 69.5 mAP with 81% recall, YOLOv2 with anchor boxes achieves 69.2 mAP with 88% recall, demonstrating enhanced detection capability despite slight mAP reduction.

7.4 Dimension Clusters via K-Means

YOLOv2 employs k-means clustering to determine optimal anchor box dimensions, addressing limitations of manually specified priors. Traditional k-means clustering with Euclidean distance metrics disproportionately weights larger boxes due to their greater spatial extent. YOLOv2 instead utilizes an Intersection over Union (IoU)-based distance metric:

$$d(\text{box}, \text{centroid}) = 1 - \text{IoU}(\text{box}, \text{centroid}) \tag{9}$$

This IoU-based distance metric is scale-invariant, ensuring equal consideration of boxes regardless of size. Through empirical evaluation, YOLOv2 selects $k = 5$ anchor boxes as an optimal trade-off between model complexity and detection recall. This clustering approach enables the network to initialize with appropriate bounding box priors that align with the statistical distribution of objects in the training dataset.

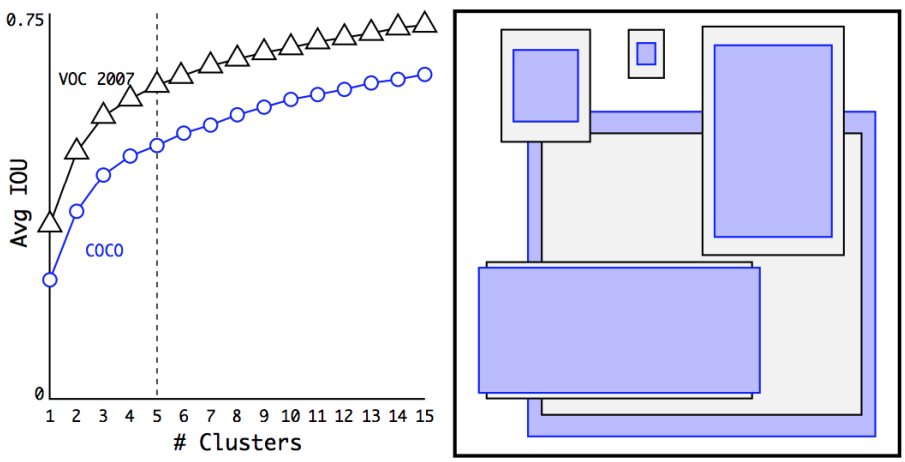


Figure 5: Clustering

7.5 Direct Location Prediction

Early implementations of anchor box mechanisms suffered from training instability and extended convergence times due to random initialization. YOLOv2 addresses this limitation through direct location prediction that constrains coordinate predictions to logical ranges. The system predicts bounding box coordinates relative to grid cell locations, with ground truth values constrained to the interval [0, 1]. This constraint is enforced through logistic activation functions that restrict network predictions to this bounded range.

The network predicts five bounding boxes at each cell within the output feature map, with each prediction comprising coordinate offsets, dimensions, objectness score, and class probabilities. This constrained prediction mechanism stabilizes training and accelerates convergence by preventing unbounded coordinate predictions that can destabilize optimization.

7.6 Fine-Grained Feature Integration

While 13×13 feature maps provide sufficient resolution for detecting large objects, they prove suboptimal for localizing smaller objects. YOLOv2 addresses this limitation through a passthrough layer that integrates higher-resolution features from earlier network stages. Specifically, the system extracts features from an earlier layer with 26×26 resolution and concatenates them with deeper, lower-resolution features.

The concatenation mechanism stacks adjacent features into different channels rather than spatial locations, transforming a $26 \times 26 \times 512$ feature map into a $13 \times 13 \times 2048$ representation. This fine-grained feature integration yields approximately 1% performance improvement, particularly enhancing detection of small objects.

7.7 Multi-Scale Training

YOLOv2 implements a dynamic multi-scale training strategy that randomly resizes input images during training. Every 10 batches, the network selects a new input dimension from a predefined set of resolutions. This approach serves as a powerful data augmentation technique that enhances model robustness to scale variations without requiring additional training data.

The convolutional architecture of YOLOv2 permits variable input dimensions, with the constraint that resolutions must be multiples of 32 to maintain consistent downsampling. This flexibility enables the network to learn scale-invariant representations while providing a practical mechanism for trading off between detection speed and accuracy: lower resolutions enable faster inference but reduce precision, while higher resolutions improve accuracy at computational cost.

8 Training Methodology

8.1 Classification Network Training

YOLOv2 incorporates a comprehensive classification training phase using the standard ImageNet 1000-class dataset. The training regimen spans 160 epochs and utilizes stochastic gradient descent with specific hyperparameters:

- Initial learning rate: 0.1
- Polynomial learning rate decay with power of 4
- Weight decay: 0.0005
- Momentum: 0.9
- Framework: Darknet neural network framework

During classification training, standard data augmentation techniques are applied to enhance model generalization. This initial classification training establishes robust feature representations that subsequently transfer to detection tasks.

8.2 Detection Network Adaptation

Following classification training, the network undergoes modification for detection tasks through architectural adjustments:

- Removal of the final convolutional layer
- Addition of three 3×3 convolutional layers with 1024 filters each
- Incorporation of a final 1×1 convolutional layer for detection output

These modifications transform the classification network into a detection architecture while preserving learned feature representations. The detection-specific layers are then trained on detection datasets to learn bounding box coordinate prediction, objectness estimation, and multi-class classification.

9 Hierarchical Classification Mechanism

9.1 Joint Training Strategy

YOLOv2 implements a novel mechanism for jointly training on both classification and detection data, enabling the system to leverage both labeled and partially labeled datasets. When processing detection-labeled images, the network learns detection-specific information including bounding box coordinate prediction and objectness estimation. When processing classification images, the network backpropagates loss only through the classification components of the architecture.

This joint training approach enables YOLOv2 to benefit from the extensive labeled classification data available in ImageNet while simultaneously learning precise localization from detection datasets.

9.2 WordNet-Based Hierarchy

YOLOv2 organizes its extensive classification capability through a hierarchical structure derived from WordNet, a lexical database that structures concepts and their relationships as a directed graph. The system constructs a hierarchical tree from ImageNet concepts, enabling multi-granular classification.

The hierarchical classification mechanism operates as follows:

- **Concept Organization**: ImageNet labels are mapped to WordNet synsets (sets of cognitive synonyms)
- **Graph Construction**: A directed graph is built from WordNet relationships, with edges representing semantic connections
- **Tree Formation**: A hierarchical tree is constructed from the graph, with broader categories at higher levels and specific subcategories at lower levels
- **Multi-Level Prediction**: During inference, the network can predict at multiple hierarchical levels, providing both specific and general classifications

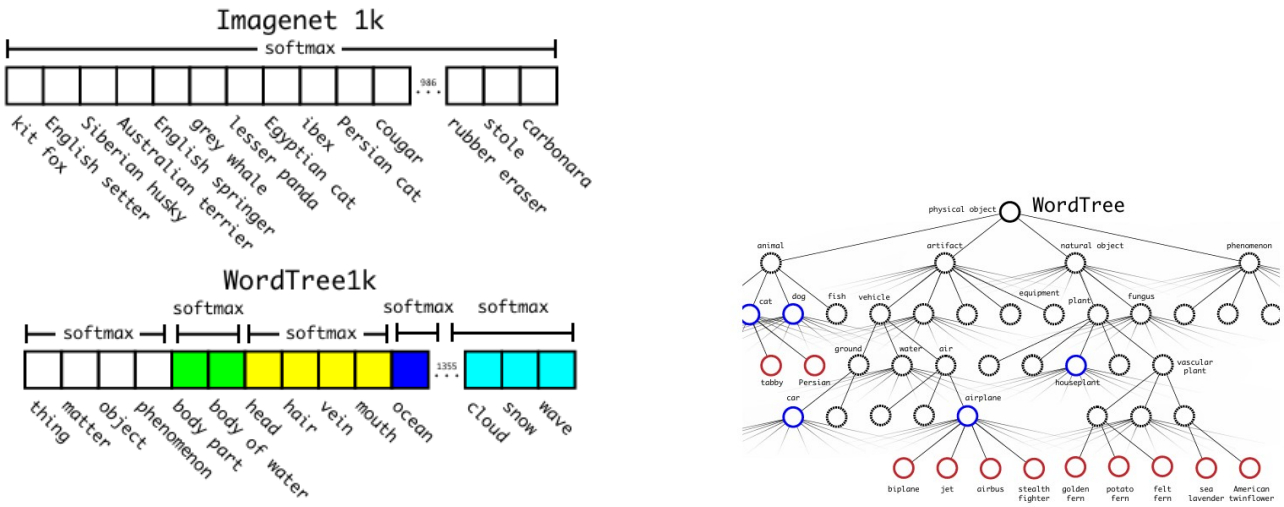
This hierarchical approach enables YOLOv2 to detect over 9,000 object categories by organizing them in a structured taxonomy that reflects real-world semantic relationships. The system can predict both specific object types (e.g., "German Shepherd") and their broader categories (e.g., "dog", "mammal", "animal") simultaneously, providing rich semantic context for detected objects.

10 Performance Characteristics

10.1 Detection Capabilities

YOLOv2 demonstrates significant improvements over its predecessor across multiple dimensions:

- **Category Coverage**: Detection capability expanded from 20 object categories to over 9,000



(a) Imagenet and Wordtree Comparision

(b) WordTree Classification Method

Figure 6: Hierarchial Classification

- **Localization Accuracy**: Enhanced bounding box precision through anchor box mechanisms and direct location prediction
- **Small Object Detection**: Improved detection of smaller objects through fine-grained feature integration
- **Scale Robustness**: Enhanced performance across varying object scales through multi-scale training
- **Training Stability**: Reduced training instability and accelerated convergence through batch normalization

10.2 Computational Efficiency

Despite its enhanced capabilities, YOLOv2 maintains real-time performance characteristics:

- **Inference Speed**: 40-90 frames per second depending on input resolution and hardware configuration
- **Model Size**: Approximately 200MB for the full YOLO 9000 model
- **Memory Efficiency**: Optimized memory usage through convolutional architecture and parameter sharing
- **Scalability**: Efficient scaling across multiple GPU configurations for accelerated training

YOLOv3

11 Introduction

YOLOv3 represents the third major iteration in the YOLO object detection architecture series, introducing incremental but significant improvements over its predecessors while preserving the fundamental design philosophy of unified, single-stage detection. The system demonstrates exceptional efficiency, processing 320×320 resolution images in 22 milliseconds while achieving 28.2 mean Average Precision (mAP), establishing an optimal balance between computational efficiency and detection accuracy. This technical document provides a comprehensive specification of the YOLOv3 architecture, detailing its enhanced bounding box prediction mechanism, multi-label classification approach, multi-scale detection strategy, and the novel Darknet-53 feature extraction backbone that collectively contribute to its improved performance characteristics.

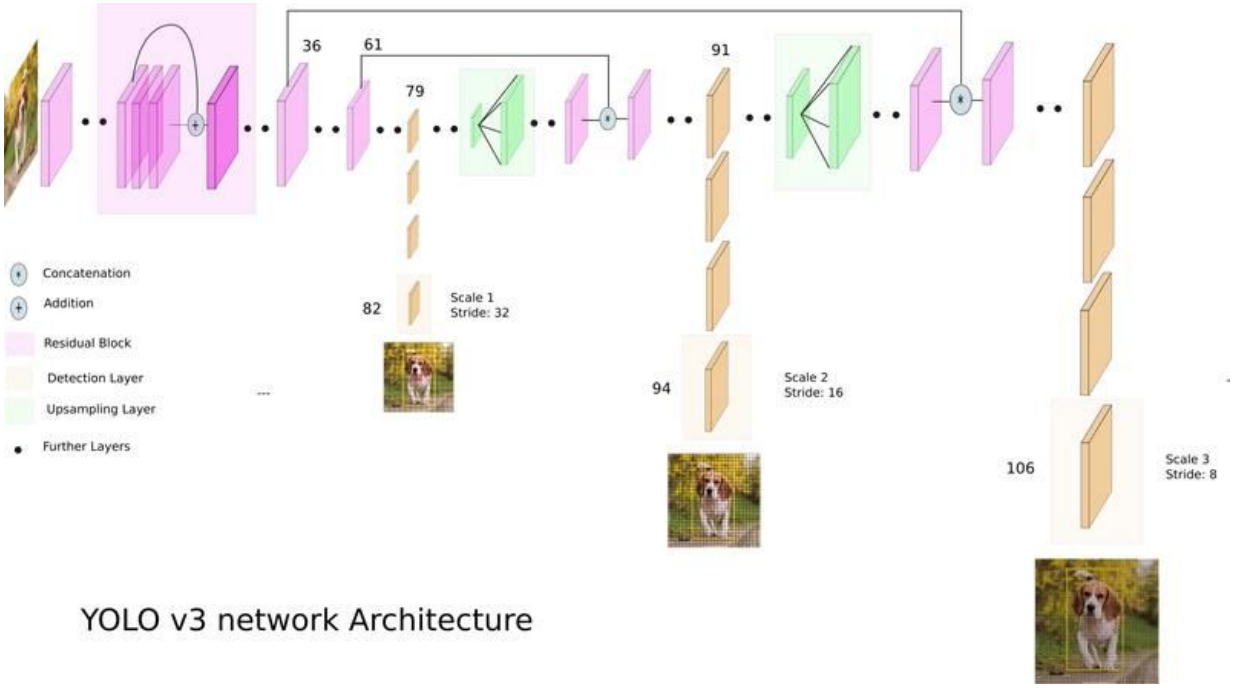


Figure 7: Convolutional Architecture with anchor boxes

12 Bounding Box Prediction Mechanism

12.1 Logistic Regression for Objectness Scoring

YOLOv3 implements a refined bounding box prediction system that employs logistic regression to compute objectness scores for each candidate bounding box. This approach represents a departure from traditional methods that utilize predefined threshold values, instead learning the optimal discrimination boundary between object presence and absence.

The objectness score s_{obj} for a bounding box b is computed through logistic regression as:

$$s_{\text{obj}}(b) = \sigma(w^T \cdot f(b) + \beta) \tag{10}$$

where σ denotes the sigmoid activation function $\sigma(x) = \frac{1}{1+e^{-x}}$, w represents learned weight parameters, $f(b)$ encodes the feature representation extracted for bounding box b, and β is the bias term.

12.2 Responsibility Assignment Mechanism

YOLOv3 incorporates a sophisticated responsibility assignment mechanism that determines which bounding box prior assumes primary responsibility for detecting each ground truth object. The system follows a rigorous assignment protocol:

- **Primary Responsibility Assignment**: A bounding box prior b_i receives primary responsibility for a ground truth object o_j if and only if the Intersection over Union (IoU) between b_i and o_j exceeds the IoU between o_j and any other bounding box prior. Formally:

$$R(b_i, o_j) = \begin{cases} 1, & \text{if IoU}(b_i, o_j) > \max_{k \neq i} \text{IoU}(b_k, o_j) \\ 0, & \text{otherwise} \end{cases} \tag{11}$$

  When this condition is satisfied, the corresponding objectness target for b_i is set to 1, indicating confident object presence.

- **Ignore Threshold Mechanism**: For bounding box priors that exhibit moderate overlap with ground truth

objects but do not achieve primary responsibility, the system implements an ignore threshold. If the IoU between a bounding box prior b_i and any ground truth object exceeds a predefined threshold τ_{ignore} (typically set to 0.5) but b_i is not the primary responsible box, the prediction is ignored during loss computation:

$$I(b_i) = \begin{cases} \text{True}, & \text{if } \exists o_j : \text{IoU}(b_i, o_j) > \tau_{\text{ignore}} \wedge R(b_i, o_j) = 0 \\ \text{False}, & \text{otherwise} \end{cases} \tag{12}$$

This mechanism prevents conflicting gradient signals during training by excluding ambiguous predictions from the optimization process.

12.3 Coordinate Prediction through Offset Parameterization

YOLOv3 employs an offset-based parameterization for bounding box coordinate prediction that enhances training stability and localization accuracy. The system predicts bounding box dimensions relative to predetermined cluster centroids derived from k-means clustering on the training dataset.

Let c_w and c_h represent the width and height of a cluster centroid. The network predicts scaling factors t_w and t_h for width and height respectively, with the final bounding box dimensions computed as:

$$b_w = c_w \cdot e^{t_w}, \quad b_h = c_h \cdot e^{t_h} \tag{13}$$

The exponential transformation ensures positive width and height values while allowing the network to learn multiplicative adjustments to the anchor dimensions.

For center coordinate prediction, the system employs a grid-relative parameterization. Given a grid cell with top-left coordinates (c_x, c_y) in the feature map, the bounding box center coordinates (b_x, b_y) are computed as:

$$b_x = c_x + \sigma(t_x), \quad b_y = c_y + \sigma(t_y) \tag{14}$$

where t_x and t_y are network predictions transformed through sigmoid activation σ, constraining the center offset to the range $(0, 1)$ and ensuring that predicted centers remain within the boundaries of their respective grid cells. This constrained prediction mechanism prevents unstable training behavior observed in earlier anchor-based detection systems.

13 Class Prediction System

13.1 Multi-Label Classification Framework

YOLOv3 addresses the fundamental limitation of mutually exclusive classification by implementing a multi-label classification framework capable of detecting objects that simultaneously belong to multiple semantic categories. This approach proves particularly valuable for real-world scenarios where objects naturally exhibit multiple categorical affiliations.

The system replaces the traditional softmax-based classification layer with independent logistic classifiers for each object category. For C distinct classes, the network produces C independent binary predictions, each estimating the probability that the detected object belongs to the corresponding class.

13.2 Independent Logistic Classifiers

Each class prediction in YOLOv3 is generated by an independent logistic regression unit that operates without explicit competition between classes. The probability p_i that a detected object belongs to class i is computed as:

$$p_i = \sigma(\theta_i^T \cdot f + \beta_i) \tag{15}$$

where σ denotes the sigmoid function, θ_i represents the learned weight vector for class i, f is the feature representation extracted for the bounding box, and β_i is the class-specific bias term.

The independence between classifiers enables the system to produce multiple high-confidence predictions for a single object when appropriate, reflecting real-world multi-label relationships.

13.3 Binary Cross-Entropy Loss Formulation

During training, YOLOv3 employs binary cross-entropy loss for each class prediction, optimizing each classifier independently. For a single bounding box prediction, the classification loss $\mathcal{L}_{\text{class}}$ is computed as:

$$\mathcal{L}_{\text{class}} = -\sum_{i=1}^{C} [y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i)] \tag{16}$$

where $y_i \in \{0, 1\}$ indicates ground truth membership in class i, and p_i is the predicted probability.

This loss formulation provides several advantages over traditional softmax-based approaches:

- **Multi-Label Compatibility**: The loss function naturally accommodates objects belonging to multiple classes simultaneously.
- **Class Imbalance Mitigation**: Each classifier can be independently balanced through weighting schemes or sampling strategies.
- **Gradient Independence**: Gradients for each class prediction do not interfere with one another, enabling more stable optimization.
- **Interpretable Confidence Scores**: Predicted probabilities provide direct, interpretable confidence measures for each class independently.

14 Multi-Scale Detection Architecture

14.1 Feature Pyramid Network Integration

YOLOv3 incorporates a multi-scale detection strategy inspired by Feature Pyramid Networks (FPN), enabling the system to detect objects across a wide range of sizes within a single forward pass. The architecture generates predictions at three distinct scales, each specialized for objects of particular size ranges while sharing computational resources through feature reuse.

The system's final feature extractor layer produces a three-dimensional tensor encoding bounding box coordinates, objectness scores, and class prediction. This design enables efficient computation while maintaining detection capability for large objects.

14.2 Upsampling and Feature Concatenation

To enhance detection performance for medium and small objects, YOLOv3 implements a feature upsampling and concatenation mechanism that combines semantic information from deeper layers with spatial detail from earlier layers.

The process operates through the following computational steps:

1. **Feature Extraction**: The network extracts feature maps from two distinct layers within the backbone architecture: a deeper layer providing rich semantic information and an earlier layer containing finer spatial details.
2. **Upsampling Operation**: Features from the deeper layer undergo $2\times$ upsampling through bilinear interpolation or transposed convolution, increasing their spatial resolution to match that of the earlier layer.

3. **Channel-Wise Concatenation**: The upsampled features and earlier layer features are concatenated along the channel dimension, producing a combined feature representation that integrates both semantic context and spatial precision.
4. **Convolutional Processing**: Additional convolutional layers process the concatenated features to learn effective representations for detection at the current scale.

14.3 Recursive Scale Prediction

The multi-scale detection process operates recursively to generate predictions at three distinct resolutions:

- **Scale 1 (Coarsest)**: The final layer of the feature extractor directly produces predictions for large objects at the lowest spatial resolution.
- **Scale 2 (Intermediate)**: Features from Scale 1 are upsampled and concatenated with features from an intermediate backbone layer, with additional convolutional processing generating predictions for medium-sized objects.
- **Scale 3 (Finest)**: Features from Scale 2 undergo further upsampling and concatenation with earlier backbone features, producing predictions for small objects at the highest spatial resolution.

This hierarchical approach ensures that each scale specializes in detecting objects within a particular size range while benefiting from features optimized for different receptive fields and semantic abstraction levels.

15 Darknet-53 Feature Extractor

15.1 Architectural Hybridization

YOLOv3 introduces Darknet-53, a novel feature extraction backbone that hybridizes architectural elements from YOLOv2's Darknet-19 with modern residual network principles.

The architecture comprises successive 3×3 and 1×1 convolutional layers organized in a pattern that maximizes feature extraction efficiency while minimizing parameter count.

15.2 Layer Composition and Connectivity

Darknet-53 consists of 53 convolutional layers organized into five major stages, with spatial downsampling occurring at transitions between stages. The network architecture follows this general pattern:

1. **Initial Convolutional Block**: A 3×3 convolutional layer with 32 filters processes the input image, followed by batch normalization and Leaky ReLU activation.
2. **Residual Stage 1**: Multiple residual blocks, each containing two 3×3 convolutional layers with shortcut connections, progressively increase channel depth while maintaining spatial resolution.
3. **Downsampling Transition**: A 3×3 convolutional layer with stride 2 reduces spatial dimensions by half while doubling channel count.
4. **Residual Stage 2-5**: Four additional stages follow similar patterns, with increasing channel counts and decreasing spatial resolutions at each transition.

The residual blocks employ identity shortcut connections that bypass two convolutional layers, mathematically expressed as:

$$F(x) = \mathcal{H}(x) + x \tag{17}$$

where x represents the input to the residual block, $\mathcal{H}(x)$ denotes the output of the two convolutional layers, and $F(x)$ is the final block output. This formulation mitigates the vanishing gradient problem and enables effective training of deep networks.

15.3 Computational Efficiency Metrics

Darknet-53 achieves exceptional computational efficiency measured in Floating Point Operations per Second (FLOPS), establishing it as one of the most efficient feature extraction backbones for real-time detection systems. The network's design principles yield several efficiency advantages:

- **Optimal Filter Sizes**: The exclusive use of 3×3 and 1×1 convolutional kernels maximizes parameter efficiency while maintaining effective receptive fields.
- **Channel Dimension Management**: Strategic increases in channel count at each downsampling stage balance representational capacity with computational cost.
- **Shortcut Connection Efficiency**: Identity shortcuts introduce minimal computational overhead while providing substantial training benefits.
- **Activation Function Optimization**: Leaky ReLU activations provide non-linearity with minimal computational cost compared to more complex alternatives.

Comparative analyses demonstrate that Darknet-53 achieves higher FLOPS efficiency than contemporary backbones with similar depth, enabling YOLOv3 to maintain real-time performance while improving detection accuracy.

16 Performance Characteristics and Benchmark Results

16.1 Inference Speed and Accuracy Trade-off

YOLOv3 establishes an optimal balance between inference speed and detection accuracy, processing 320×320 resolution images in 22 milliseconds while achieving 28.2 mean Average Precision (mAP) on the COCO dataset benchmark. This performance profile represents a significant advancement over previous YOLO iterations and competitive performance relative to contemporaneous detection systems.

The system's speed-accuracy trade-off can be characterized by the following relationships:

- **Resolution Scaling**: Input resolution significantly impacts both inference time and detection accuracy, with higher resolutions improving mAP at the cost of increased computation.
- **Architectural Efficiency**: The Darknet-53 backbone achieves near-linear scaling of inference time with input resolution due to its optimized layer composition.
- **Multi-Scale Overhead**: The three-scale detection architecture introduces approximately 15% computational overhead compared to single-scale detection but yields 30-40% improvement in small object detection accuracy.

References

[1] Redmon, J., et al. (2016). *You Only Look Once: Unified, Real-Time Object Detection.* Available at: `arxiv.org`

[2] Redmon, J., and Farhadi, A. (2017). *YOLO9000: Better, Faster, Stronger.* Available at: `arxiv.org`

[3] Redmon, J., and Farhadi, A. (2018). *YOLOv3: An Incremental Improvement.* Available at: `arxiv.org`

[4] Redmon, J. *YOLO: Real-Time Object Detection on Darknet.* Official Project Page. Available at: `pjreddie.com` (Accessed: Dec 26, 2025).

[5] Ultralytics (2025). *Ultralytics YOLO Docs: Real-Time Object Detection Guide.* Official Documentation. Available at: `docs.ultralytics.com`

[6] V7 Labs (2024). *YOLO Object Detection: The Definitive Guide.* Technical Blog. Available at: `www.v7labs.com`

[7] DataCamp (2024). *YOLO Object Detection Explained: Architecture and Evolution.* Available at: `www.datacamp.com`

[8] Ayalew, A. / Towards Data Science (2020). *YOLO v3 Explained.* Available at: `towardsdatascience.com`

[9] Computer Vision Guide (2025). *YOLO Series Training and Implementation* [Video Playlist]. YouTube. Available at: `youtube.com`